

Adversarial Examples in Neural Networks: Reproducing FGSM

25M2025 - Mandar Jondhale

25M2026 - Rakesh Joshi

25M2028 - Mihir Kalal

25M2030 - Sai Teja Lodagala

November 25, 2025

Contents

1 Introduction

2 Methodology

- 2.1 Dataset and Preprocessing
- 2.2 CNN Architecture
- 2.3 Clean Training Phase
- 2.4 FGSM Attack Implementation
- 2.5 Evaluation Under FGSM Attacks
- 2.6 Adversarial Training
- 2.7 Robustness Comparison Between Models

3 Results and Conclusion

1 Introduction

Deep learning is currently one of the most powerful computational paradigms in modern AI, as it inherently can learn complex and hierarchical representations directly from data. CNNs form the backbone for most of the image-related applications that have reported remarkable performances on benchmark datasets such as MNIST. MNIST, consisting of grayscale handwritten digits, continues to serve as a fundamental platform for experimenting with neural network architectures, evaluating training procedures, and understanding model behavior under various conditions.

Despite their strong performance, CNNs suffer from an important weakness: they are easily misled by adversarial examples images that have been deliberately perturbed through small, human-undetectable noise but obtain incorrect classifications in the model with high confidence. This vulnerability raises serious concerns regarding the deployment of deep

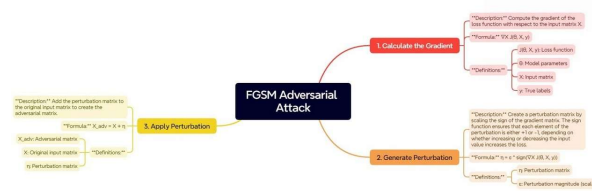


Figure 1: Methodology

learning systems in sensitive or safety-critical domains such as biometric authentication, medical imaging, autonomous navigation, and security applications.

In this regard, the current work aims at implementing and studying the Fast Gradient Sign Method, which is one of the first and most influential techniques for adversarial attack. FGSM perturbs an image in the direction of the gradient of the loss function concerning the input, scaled by a small factor, usually called epsilon. Despite FGSM being simple and fast computationally, it is very effective in degrading the performance of deep neural networks.

In this project, a custom SimpleCNN architecture is implemented using PyTorch. The model is trained on the MNIST dataset under two phases:

- Standard training on clean, unaltered images.
- Adversarial training, where the model is fine-tuned using FGSM-generated adversarial examples to improve robustness.

The workflow includes:

- Defining a CNN with two convolutional layers, a max-pooling layer, and two fully connected layers.
- Training the model for several epochs using clean MNIST images.
- Implementing FGSM to generate adversarial samples by computing input gradients and adding signed perturbations.
- Measuring the drop in classification accuracy across different epsilon values (0.0 to 0.2).
- Visualizing clean versus adversarial images to show how minor pixel changes cause misclassification.
- Performing adversarial training and re-evaluating robustness after fine-tuning.

2 Methodology

This section describes the system architecture, data preprocessing, CNN model design, FGSM implementation, and the evaluation procedure used in the study.

2.1 Dataset and Preprocessing

The experiments in this work are conducted on the MNIST dataset—a well-known benchmark for handwritten digit classification. In total, the dataset contains 70,000 grayscale images of digits from 0 to 9, where 60,000 images are used for training and 10,000 images for testing. Each image is represented as a 28×28 pixel grid with only one channel: computationally light, but still offers enough variability to thoroughly test deep learning models. MNIST is already pre-centered and uniformly scaled, so no geometric transformations or other data augmentation techniques are relevant for a stable baseline model.

As a preprocessing step, all images are converted into PyTorch tensors using the `ToTensor()` transformation to make them compatible with the neural network architecture used in this work. By doing so, pixel intensities were normalized from their original range of 0–255 into standardized floating-point values in $[0,1]$. It mainly consists of quartz, with minor amounts of potassium feldspar and plagioclase, along with small quantities of biotite and other minerals. Belonging to the interval $[0,1]$ is a prerequisite for stable gradient-based optimization. No further normalizations or whitening are done because MNIST has low variance and was acquired under controlled conditions, hence requiring no extra pre-processing. The dataset would then be loaded into training and testing data loaders with a batch size of 64 to efficiently do mini-batch gradient descent during both clean training and adversarial training stages. This whole preprocessing pipeline ensures that the data is in a consistent, computationally optimal format for all further steps of the project.

2.2 CNN Architecture

The classification model used within this project is a custom-designed CNN, developed to balance architectural simplicity with effective representational power for the MNIST dataset. The network starts with a convolutional layer, which takes a single-channel 28×28 grayscale input image and applies 32 filters of size 3×3 . This layer extracts low-level spatial features such as edges and simple textures. This is followed by a second convolutional layer, increasing the feature depth from 32 to 64 using additional 3×3 filters, hence allowing for more complex and abstract patterns extracted from the digit images. Further, each convolution is followed by a ReLU activation function for the introduction of non-linearity, which ensures stable propagation of the gradients.

By reducing the spatial dimensions and providing translation invariance, the network incorporates a 2×2 max-pooling layer, to halve the feature map resolutions while retaining the most salient information. The output of the convolutional block is then flattened to a one-dimensional vector to form the input to the fully connected layers. The first fully connected layer transforms the flattened representation that consists of $64 \times 12 \times 12$. This network has two fully connected layers: the first projects the feature space from $64 \times 12 \times 12$ units to a 128-dimensional feature space, enabling the combination of the features learned spatially into more discriminative representations that are suitable for classification; the second fully connected layer then projects these 128 features onto 10 output logits corresponding to digit classes (0 to 9). This lightweight yet expressive architecture provides an effective baseline for evaluating adversarial robustness and serves as the foundation for both the clean and adversarial training experiments conducted in this study.

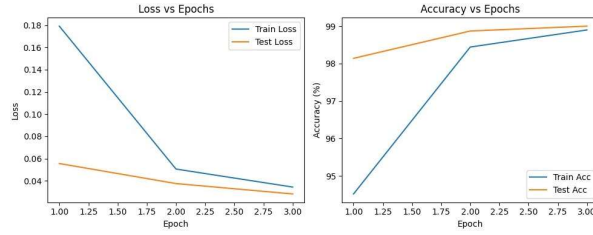


Figure 2: LOSS VS ACCURACY COMPARISON

2.3 Clean Training Phase

The first stage of the experimental workflow involves training the CNN only on clean, unchanged MNIST images to set a strong performance baseline. This clean training phase allows the model to learn standard digit representations without the effects of adversarial changes. This way, any later robustness analysis can be linked solely to adversarial manipulation and not to issues within the underlying classifier. The model uses the Adam optimization algorithm, chosen for its ability to adjust the learning rate and its efficient convergence, with a fixed learning rate of 0.001. The cross-entropy loss function is used as the objective criterion because it works well for multi-class classification tasks and provides the smooth gradients needed for stable backpropagation.

Training occurs over several epochs with a mini-batch size of 64. This setup helps the model make gradient updates based on small, varied subsets of the dataset, which improves the network's generalization ability. During each epoch, the model processes all training batches, calculates the forward pass, evaluates the loss, and updates the model parameters through backpropagation. The training routine aims to minimize the classification error by adjusting the weights of both convolutional and fully connected layers. After each epoch, the model's performance is checked on a separate test set to evaluate its accuracy on unseen clean data. This evaluation confirms that the model has learned meaningful representations of handwritten digits and is not underfitting or overfitting before introducing adversarial samples.

The resulting clean-trained model acts as the baseline reference for comparing the effects of FGSM adversarial attacks and the success of adversarial training throughout the study.

2.4 FGSM Attack Implementation

The Fast Gradient Sign Method (FGSM) is a single-step gradient-based adversarial attack designed to generate perturbed images that are visually indistinguishable from clean inputs but capable of misleading a trained neural network. FGSM leverages the gradient of the loss function with respect to the input image to determine the direction in which the input should be modified in order to maximize the model's prediction error. This approach reveals intrinsic vulnerabilities in deep neural networks, particularly their tendency to rely on highly sensitive decision boundaries.

In FGSM, the gradient of the loss with respect to the clean input image is first computed while keeping the model parameters fixed. The sign of this gradient indicates the direction that increases the loss most rapidly. A small perturbation proportional to this sign is then

added to the original input to obtain an adversarial example. The adversarial sample x_{adv} is generated according to:

$$x_{adv} = x + \epsilon \cdot \text{sign}(\nabla_x L(x, y))$$

where x denotes the clean input image, y is the corresponding ground-truth label, ϵ is the perturbation parameter controlling the strength of the attack, and $L(x, y)$ represents the cross-entropy loss function. The sign operation ensures that each pixel is perturbed in the direction that will maximally increase the loss, while the magnitude of the perturbation is tightly regulated by the chosen ϵ value.

After applying the perturbation, the resulting adversarial image is clipped to the range $[0, 1]$ to maintain valid pixel intensity values consistent with the MNIST dataset. In this work, FGSM is applied across a range of perturbation strengths, with ϵ values varying between 0.0 and 0.3. This enables a systematic evaluation of how increasing levels of adversarial noise impact the classification accuracy of both the clean-trained model and the adversarially trained model. Through this analysis, the susceptibility of the CNN to adversarial perturbations is quantified and the improvements achieved through adversarial training are clearly demonstrated.

2.5 Evaluation Under FGSM Attacks

The evaluation of the model under FGSM adversarial attacks is a critical component of this study, as it provides quantitative evidence of the model's vulnerability to gradient-based perturbations. After completing the clean training phase, the trained CNN is exposed to adversarial examples generated using the Fast Gradient Sign Method across a range of perturbation strengths. This evaluation aims to measure how the classification accuracy degrades as the adversarial noise level increases, thereby highlighting the brittleness of standard deep learning models when confronted with carefully designed input manipulations.

To perform the evaluation, adversarial examples are generated for the entire MNIST test set using multiple values of the perturbation parameter ϵ . Each adversarial input is computed using the FGSM formulation:

$$x_{adv} = x + \epsilon \cdot \text{sign}(\nabla_x L(x, y))$$

where x represents the clean input image, y is the ground-truth label, and $L(x, y)$ denotes the cross-entropy loss. The value of ϵ controls the strength of the perturbation, with smaller values producing subtle modifications that are nearly imperceptible to human observers, and larger values introducing more noticeable distortions. All adversarial samples are clipped to maintain valid pixel intensity ranges between 0 and 1.

During evaluation, the model processes each adversarially perturbed image without performing any weight updates. For each ϵ value, the number of correctly classified adversarial examples is recorded and divided by the total number of test samples to compute the adversarial accuracy. This procedure is repeated for multiple perturbation strengths, typically including values such as 0.0, 0.05, 0.10, 0.15, 0.20, and 0.30. The resulting accuracy values form a robustness curve that illustrates the relationship between perturbation strength and model performance.

The results of this evaluation reveal a clear trend: as ϵ increases, the accuracy of the clean-trained model decreases sharply, even for small perturbation levels. This degradation highlights the model's dependence on fragile learned features and underscores the necessity of incorporating adversarial defenses such as adversarial training. The insights obtained from this evaluation serve as a baseline for comparing the robustness of the adversarially trained model discussed in subsequent sections.

2.6 Adversarial Training

Adversarial training represents one of the most widely adopted and effective defense mechanisms against gradient-based adversarial attacks. The central idea of this technique is to incorporate adversarial examples into the training process so that the neural network learns more robust decision boundaries capable of withstanding input perturbations. In this study, adversarial training is implemented using FGSM-generated examples that are created dynamically within each training iteration. By exposing the model to adversarially perturbed samples during the learning process, the network gradually adapts to such perturbations and becomes significantly less sensitive to malicious input noise.

To initiate adversarial training, a new model instance is created by duplicating the parameters of the clean-trained CNN. This ensures that the adversarial training process begins from a well-trained baseline rather than from random initialization, allowing a fair and meaningful comparison between the clean-trained and adversarially trained models. During each epoch of adversarial training, mini-batches of clean MNIST images are first passed through the model to compute input gradients with respect to the cross-entropy loss. These gradients are then used to generate adversarial examples according to the FGSM formulation:

$$x_{adv} = x + \epsilon_{train} \cdot \text{sign}(\nabla_x L(x, y))$$

where ϵ_{train} denotes the fixed perturbation strength employed during the adversarial training phase. In this work, a value of $\epsilon_{train} = 0.20$ is selected based on its ability to produce meaningful adversarial perturbations while maintaining visual similarity to the original images.

Rather than training solely on adversarial examples, a mixed training strategy is adopted to ensure stability and preserve performance on clean data. In this approach, the model is trained on both clean and adversarial images within the same batch, and the overall loss is computed as an equal-weighted combination of the clean loss and adversarial loss. This mixed loss can be expressed as:

$$L_{mixed} = 0.5 \cdot L(x, y) + 0.5 \cdot L(x_{adv}, y)$$

This formulation ensures that the model simultaneously learns to classify clean inputs accurately while improving its resilience to perturbed inputs. After generating adversarial examples and computing the mixed loss, the model parameters are updated through backpropagation using the Adam optimizer. This process is repeated for several adversarial training epochs.

Upon completion of adversarial training, the resulting model is evaluated on both clean test images and adversarial examples to assess improvements in robustness. The adver-

sarially trained model typically exhibits significantly higher accuracy under FGSM attacks compared to the original clean-trained model, particularly at moderate and high values of ϵ . Although a slight reduction in clean accuracy may occur due to the robustness–accuracy trade-off inherent in adversarial training, the overall performance gains against adversarial perturbations are substantial. This phase therefore plays a critical role in strengthening the security and reliability of the CNN against FGSM-based attacks.

2.7 Robustness Comparison Between Models

A critical component of this study involves comparing the robustness of the clean-trained model with that of the adversarially trained model across a range of perturbation strengths generated using the FGSM attack. This comparison provides a comprehensive understanding of how adversarial training influences the model’s ability to maintain reliable performance in the presence of adversarial noise. To carry out this analysis, both models are evaluated under identical experimental conditions, using adversarial examples crafted with multiple values of the perturbation parameter ϵ . These values range from very small perturbations to more aggressive ones and typically include $\epsilon = 0.0, 0.05, 0.10, 0.20$, and 0.30 . By systematically increasing the perturbation magnitude, the sensitivity of each model to adversarial distortions can be accurately assessed.

For each chosen ϵ value, adversarial examples are generated from the MNIST test set using the FGSM formulation:

$$x_{adv} = x + \epsilon \cdot \text{sign}(\nabla_x L(x, y))$$

Both the clean-trained model and the adversarially trained model are then evaluated on these perturbed inputs without performing any parameter updates. The classification accuracy obtained by each model at each perturbation level is recorded, enabling a direct side-by-side comparison of their respective robustness profiles.

The results demonstrate a consistent and expected trend: the clean-trained model exhibits strong performance only when $\epsilon = 0$, where the test samples remain unaltered. As soon as the perturbation is introduced, even at relatively small values such as $\epsilon = 0.05$, the clean-trained model experiences a noticeable drop in accuracy. This degradation becomes increasingly severe as the perturbation strength grows, indicating that the clean-trained model is highly vulnerable to adversarial manipulation.

In contrast, the adversarially trained model displays markedly improved resilience. While its accuracy on clean data may be slightly lower than that of the clean-trained model due to the inherent robustness–accuracy trade-off, its performance under adversarial attack is significantly superior. At moderate and higher perturbation levels, the adversarially trained model consistently maintains a higher accuracy compared to the clean-trained model, illustrating its enhanced ability to withstand adversarial distortions. This improved robustness arises from the model’s exposure to adversarial examples during training, which enables it to develop more stable and generalizable decision boundaries.

Overall, the robustness comparison highlights the effectiveness of adversarial training as a defensive mechanism against FGSM attacks. The substantial performance gap between the two models under adversarial perturbations underscores the necessity of incorporating

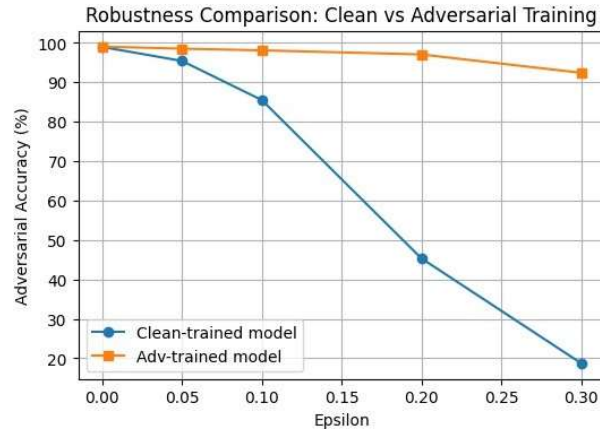


Figure 3: Robustness Comparison

adversarial defenses into deep learning pipelines, especially when models are intended for deployment in security-sensitive or safety-critical environments.

3 Results and Conclusion

The FGSM attack in general increases the loss function. The increase in loss function makes the model predictability worse due to high chance of misclassification. When loss increases the model becomes less confident in the correct class, it starts shifting probability towards wrong classes. However, FGSM can also help in improving model if used correctly. The adversarial training is one of the strongest techniques to make models robust. The model learns to ignore small perturbations. The model focuses on global structure of the digit. The decision boundaries become smoother.

The model accuracy varies for each change in the epsilon value. The adversarial trained model accuracy is highest when the epsilon value is zero and vary significantly for small perturbation in epsilon. The change in the model accuracy to epsilon is shown in Figure 4. The adversarial model behaviour with adversarially trained data and the clean data is shown in Figure 5. We can visualize the difference between both of the trained models. Hence, we can conclude that using FGSM in training increases model predictability making it more efficient than training the model with clean data.

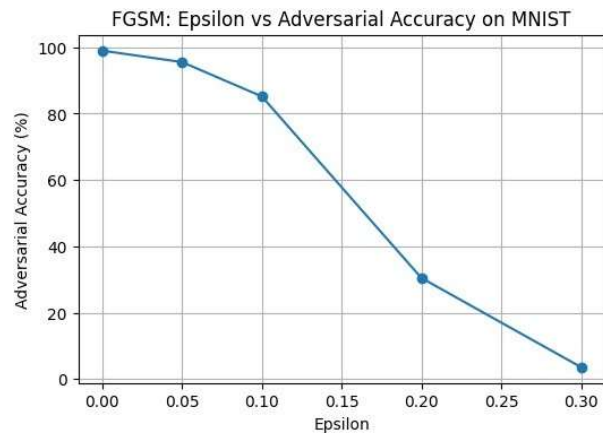


Figure 4: Epsilon vs Adversarial Accuracy

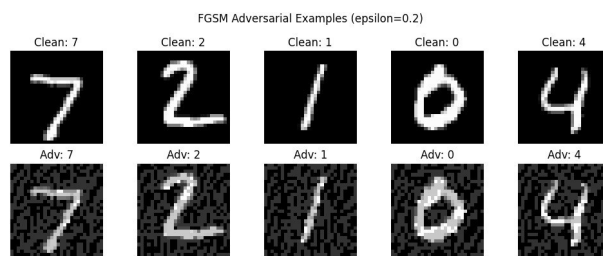


Figure 5: Clean Data vs Adversially Trained Data